

MySQL et SQL

Système de gestion de base de données

Un système de gestion de base de données, ou SGBD, est tout simplement un logiciel dont le rôle est de gérer des bases de données. Il en existe de très nombreux, et nous allons nous concentrer sur les systèmes de gestion de bases de données relationnelles, **SGBDR**. Et parmi les nombreux SGBDR (eh oui, on n'en finit pas), nous allons nous concentrer sur **MySQL**.

MySQL

Pourquoi MySQL ? Parce qu'il est le SGBDR le plus connu et le plus courant, c'est tout. Il y a de très fortes chances pour que les premières bases de données que vous manipulerez dans votre carrière soient des bases MySQL, voire éventuellement des systèmes lui ressemblant fortement (MariaDB, PostgreSQL, etc.)

MySQL stocke les données sous la forme de bases et de tables, comme nous l'avons vu précédemment. En somme, toute l'introduction de ce module peut déjà servir à comprendre MySQL dans ses grands principes : des bases de données, elles-mêmes constituées de tables, elles-mêmes constituées d'entrées, elles-mêmes constituées de colonnes.

SQL

SQL, pour « *Structured Query Language* », est un langage qui nous permet de communiquer avec MySQL. Toutes les interactions avec MySQL passent par le SQL, sous la forme de requêtes. Que vous vouliez ajouter des données, en modifier, ou simplement en lire, toutes vos demandes se font en SQL. Un peu comme un bibliothécaire à qui on ne s'adresserait qu'en latin.

Connaître ce langage fait partie du socle des compétences fondamentales d'un développeur web. On n'y échappe pas et, soyez prévenus, il vous donnera des migraines. (:

En effet, si la syntaxe du SQL est simpliste, c'est la logique qui peut être parfois difficile à acquérir. Mais personne n'a dit que développeur était un métier facile, à ce stade de la formation vous commencez à le comprendre, et avec de la pratique tout est surmontable.

CRUD

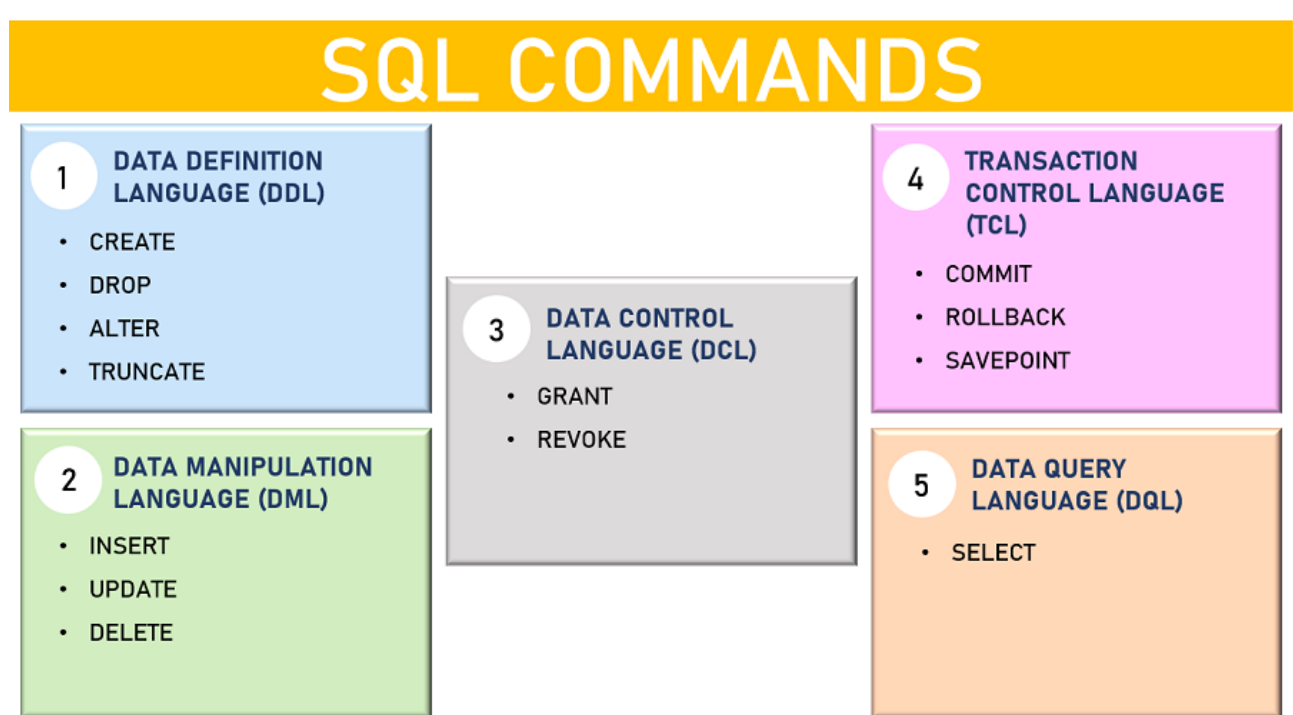
Nous allons sauter directement à la pratique : manipuler une base de données. Vos formateurs vous aideront à en mettre en place via des outils tels que **PHPMyAdmin** ou Adminer. Viendront ensuite les requêtes regroupant les quatre grandes actions que l'on peut exécuter sur un jeu de données : création, lecture, modification, suppression. En anglais : **Create, Read, Update, Delete**. CRUD.

La bonne nouvelle est que trois de ces actions sont extrêmement simples. La création, la modification et la suppression sont, la plupart du temps, des requêtes routinières. Ce sont les requêtes de lecture qui mobiliseront vraiment vos neurones et demanderont de l'entraînement.

Les chapitres suivants seront dédiés à ces actions, en commençant par la lecture.

Les types d'instructions

Globalement l'ensemble des instructions SQL peuvent se ranger dans 5 grandes catégories illustrées ci-dessous :



phpMyAdmin

phpMyAdmin est une application web open **source** utilisée pour gérer et administrer des bases de données MySQL via une interface utilisateur conviviale. Il s'agit d'un outil populaire parmi les développeurs web et les administrateurs de bases de données, car il simplifie grandement la gestion des bases de données MySQL.

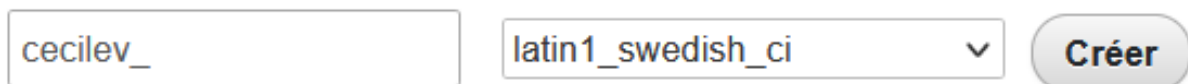
Pour accéder à phpMyAdmin sur l'ide de la 3wa, dans le menu "wecode", vous trouverez un lien vers phpMyAdmin.

Voici quelques-unes des fonctionnalités principales de phpMyAdmin :

Gestion des bases de données

Vous pouvez créer, supprimer et gérer des bases de données MySQL.

Dans la barre latérale droite, vous allez retrouver l'ensemble des bases de données que vous possédez déjà et un lien "Nouvelle base de données" pour, comme son nom l'indique, créer une nouvelle base:



The image shows a user interface for creating a new database. It consists of three main elements: a text input field on the left containing the text 'cecilev_', a dropdown menu in the middle showing 'latin1_swedish_ci' with a downward arrow, and a rounded button on the right labeled 'Créer'.

Veillez à laisser le préfixe du nom de base de données. Ceci est vrai pour l'ide de la 3wa, c'est ce préfixe qui vous donnera les droits pour administrer votre base de données. Dans un contexte autre que cet IDE, ce préfixe n'est pas à mettre.

La liste déroulante vous permettra de choisir l'encodage souhaité pour la base de données, idéalement UTF8.

Choisissez toujours *utf8mb4_general_ci* !

Gestion des tables

Vous pouvez créer, modifier et supprimer des tables au sein d'une base de données, ainsi que définir les relations entre les tables.

Vous allez ici reprendre votre MPD et chacune des tables qui le contient.

 **Nouvelle table**

Nom:

Nombre de colonnes:

Exécuter

Après avoir indiqué le nom de la table et le nombre de champs qui la composent, vous allez parvenir au détail de la table :

Nom	Type	Taille/Valeurs*	Valeur par défaut	Interclassement	Attributs	Null	Index	A...
id Choisir à partir des colonnes centrales	INT	10	Aucun(e)			☐	PRIMARY PRIMARY	☒
name Choisir à partir des colonnes centrales	VARCHAR	50	Aucun(e)			☐	---	☐
description Choisir à partir des colonnes centrales	TEXT		Aucun(e)			☐	---	☐
id_category Choisir à partir des colonnes centrales	INT	10	Aucun(e)			☐	INDEX [id_category]	☐

Voici un exemple.

En **première colonne** le nom du champ. Attention à utiliser la même règle de nommage pour l'ensemble des champs sinon vous vous y perdrez vite.

En **deuxième colonne**, le type de donnée, comme défini dans **le MPD**.

En **troisième colonne** la taille si nécessaire.

La **colonne index** permet de définir si un champ est une clé primaire ou un index (clé secondaire)

Et enfin, la **colonne AI**, permet de mettre en place l'autoincrémentation pour des champs tels que les clés primaires.

Les relations

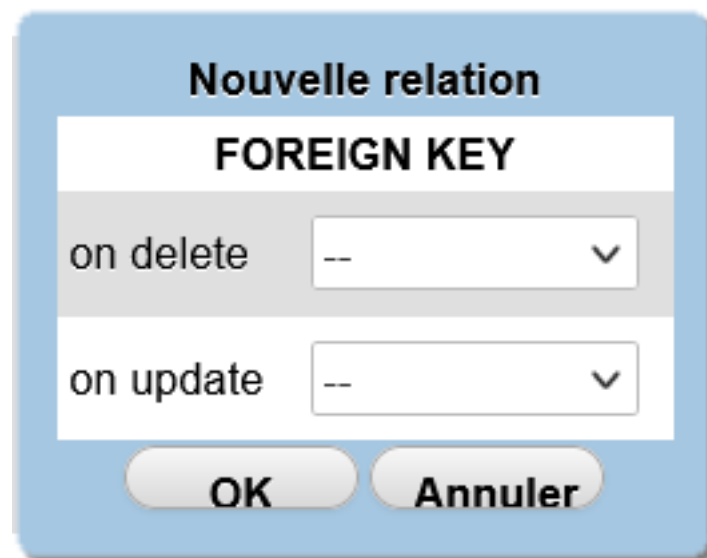
Une fois l'ensemble des tables réalisées, il faut maintenant aller dans le concepteur pour mettre en place les relations entre les différentes tables. Pour ce faire, cliquez dans la barre latérale sur le nom de la base de données, puis sur l'onglet concepteur (ou designer si votre phpMyAdmin est en anglais).

Les différentes tables apparaissent. Pour créer une relation, il faut cliquer dans le menu composé de symboles sur la gauche, sur l'item suivant :



La souris est alors suivie d'un texte "Sélectionner la clé référencée", ce qui signifie qu'il faut cliquer sur la clé primaire qui constitue la relation. Une fois que c'est chose faite, le texte change et devient "Sélectionner la clé étrangère" et donc cette fois ci, il faut cliquer sur la clé étrangère de l'autre table qui constitue la relation.

Nous allons alors définir les contraintes de la relation :



Nouvelle relation

FOREIGN KEY

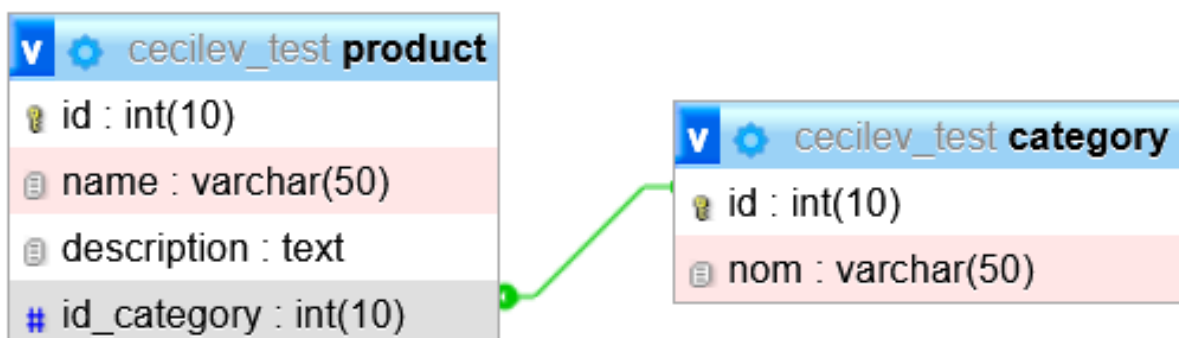
on delete -- ▾

on update -- ▾

OK Annuler

Ici, phpMyAdmin nous demande quoi faire lors de la suppression et de la mise à jour d'un élément.

Prenons l'exemple de cette relation :



La question est "Si l'on supprime une catégorie, que doit-on faire des produits qui font partie de cette catégorie ?". 4 options :

- CASCADE : signifie que en cas de suppression de catégorie, les produits de cette catégorie sont également supprimés
- SET NULL : signifie que l'id_categorie des produits de la catégorie supprimée sera passé à la valeur NULL
- NO ACTION : on ne change rien, le produit se retrouvera alors associé à une catégorie qui n'existe pas (déconseillé car la base de données n'est plus correcte ensuite)
- RESTRICT : signifie que l'on empêche la suppression de toute catégorie qui contiendrait des produits

A ce moment là, c'est à vous de définir la règle que vous souhaitez appliquer à cette relation.

Insérer des données

Lorsque vous cliquez sur le nom d'une table de la base de données, parmi les onglets de phpMyAdmin, se trouve "Insérer". En cliquant dessus, un formulaire s'ouvre vous permettant d'insérer des enregistrements.

Si vous aviez déterminé un id auto incrémenté, alors il n'est pas nécessaire de remplir ce champ. Pour les autres champs, il n'y a qu'à remplir et cliquer sur "Exécuter":

Colonne	Type	Fonction	Null	Valeur
id	int(10)			
nom	varchar(50)			Tech

☐ Ignorer

Colonne	Type	Fonction	Null	Valeur
id	int(10)			
nom	varchar(50)			Voyage

Importation et exportation de données

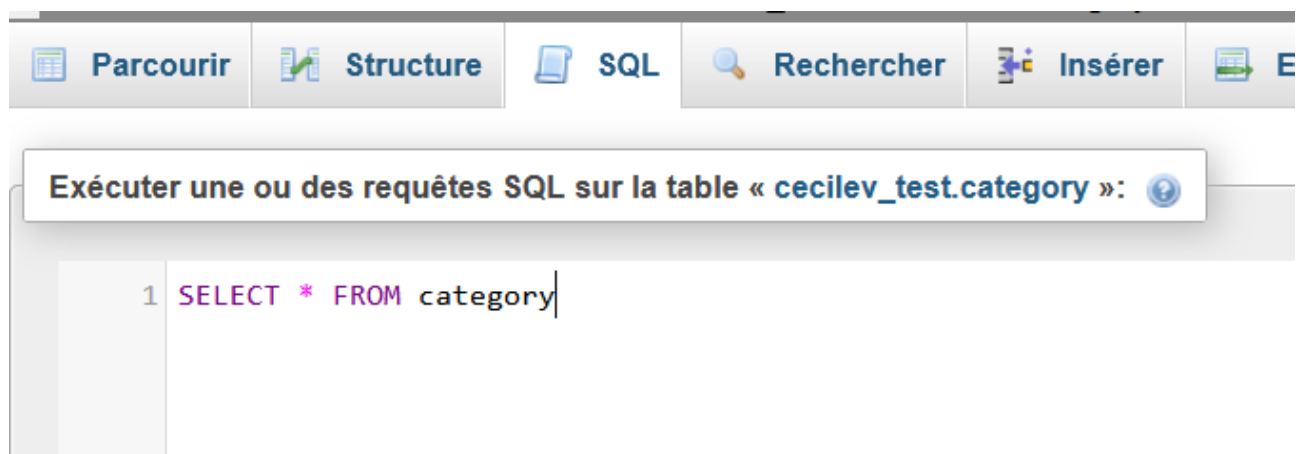
Vous pouvez importer des données depuis des fichiers SQL ou CSV dans une base de données et exporter des données vers différents formats, y compris SQL, CSV, Excel et plus encore. Il suffit d'utiliser **les interfaces** des onglets Exporter et Importer :



Exécution de requêtes SQL

phpMyAdmin offre un éditeur SQL qui vous permet d'exécuter des requêtes SQL personnalisées directement depuis l'interface.

Il vous suffit de cliquer sur l'onglet SQL et dans le champ disponible, écrire la requête que vous souhaitez exécuter :



Visualiser les données

L'onglet "Parcourir" vous permettra de visualiser les données présentes dans la table et d'agir dessus, de la modifier, de la supprimer

phpMyAdmin offre encore beaucoup de possibilités que vous découvrirez avec le temps. Certaines ne vous serviront peut être jamais.

Select

Une requête de lecture commence toujours par le mot-clé **SELECT**.

Reprenons notre liste de films :

movies

i d	year	title	directo r
1	1966	<i>Persona</i>	4
2	1997	<i>La Vie de Jésus</i>	5
3	2003	<i>Sarabande</i>	4
4	1978	<i>The Deer Hunter</i>	1
5	1976	<i>Taxi Driver</i>	2
6	1950	<i>Rashōmon</i>	3

Admettons que dans cette table nommée **movies** nous voulions récupérer les titres de tous les films sans exception :

```
SELECT title FROM movies
```

Détaillons.

Le mot-clé **SELECT**, suivi de la colonne – ou des colonnes – que nous voulons récupérer.

Puis le mot-clé **FROM**, suivi de la table à parcourir.

Traduction de la requête : « Sélectionne les titres des films ! »

La réponse de MySQL à cette requête ressemblera à ça :

title
<i>Persona</i>
<i>La Vie de Jésus</i>
<i>Sarabande</i>
<i>The Deer Hunter</i>
<i>Taxi Driver</i>
<i>Rashōmon</i>

Nous pouvons bien sûr demander plusieurs colonnes :

```
SELECT id, title FROM movies
```

On sépare les colonnes par des virgules **,**. Résultat :

i d	title
1	<i>Persona</i>
2	<i>La Vie de Jésus</i>
3	<i>Sarabande</i>
4	<i>The Deer Hunter</i>
5	<i>Taxi Driver</i>
6	<i>Rashōmon</i>

Et si nous voulons toutes les colonnes, il existe un raccourci syntaxique :

```
SELECT * FROM movies
```

Ici le caractère ***** signifie « tout ».

Résultat :

i d	year	title	director_i d
1	1966	<i>Persona</i>	4
2	1997	<i>La Vie de Jésus</i>	5
3	2003	<i>Sarabande</i>	4
4	1978	<i>The Deer Hunter</i>	1
5	1976	<i>Taxi Driver</i>	2
6	1950	<i>Rashōmon</i>	3

Ceci est la base d'une requête **SELECT**. Compliquons les choses !

WHERE

Le mot-clé **WHERE** sert à préciser des conditions. Mettons que nous voulions tous les films réalisés par Ingmar Bergman, dont l'ID est 4.

```
SELECT title FROM movies WHERE director = 4
```

Le signe **=** désigne l'égalité.

Traduction : « Sélectionne les titres des films dont le réalisateur est 4. »

Résultat :

title
<i>Persona</i>
<i>Sarabande</i>

Simple !

Tous les films réalisés après 1970 ?

```
SELECT title FROM movies WHERE year > 1970
```

Résultat :

title
<i>La Vie de Jésus</i>
<i>Sarabande</i>
<i>The Deer Hunter</i>
<i>Taxi Driver</i>

Le mot-clé **WHERE** étant une condition, il obéit à la même grammaire algorithmique qu'un **if**. Ainsi, les notions **et** et **ou** existent à travers les mots-clés **AND** et **OR**.

Si nous voulons sélectionner tous les films réalisés entre 1960 et 2000.

```
SELECT title FROM movies WHERE year >= 1960 AND year <= 2000
```

Résultat :

title
<i>Persona</i>
<i>La Vie de Jésus</i>
<i>The Deer Hunter</i>
<i>Taxi Driver</i>

Notez que cette requête peut être raccourcie grâce au mot-clé **BETWEEN** !

ORDER BY

Nous pouvons trier l'ordre des résultats grâce au mot-clé **ORDER BY**.

```
SELECT id, title FROM movies ORDER BY title
```

Résultat :

i d	title
2	<i>La Vie de Jésus</i>
1	<i>Persona</i>
6	<i>Rashōmon</i>
3	<i>Sarabande</i>
5	<i>Taxi Driver</i>
4	<i>The Deer Hunter</i>

Les films sont classés par ordre alphabétique de la colonne **title**.

```
SELECT id, title FROM movies ORDER BY year
```

Résultat :

i d	title
6	<i>Rashōmon</i>
1	<i>Persona</i>
5	<i>Taxi Driver</i>
4	<i>The Deer Hunter</i>
2	<i>La Vie de Jésus</i>
3	<i>Sarabande</i>

Les films sont classés par année. Oui, nul besoin que la colonne servant au tri soit forcément sélectionnée pour le résultat final.

Et si nous voulons, à l'inverse, classer les films du plus récent au plus ancien ?

```
SELECT id, title FROM movies ORDER BY year DESC
```

Résultat :

i d	title
3	<i>Sarabande</i>
2	<i>La Vie de Jésus</i>
4	<i>The Deer Hunter</i>
5	<i>Taxi Driver</i>
1	<i>Persona</i>
6	<i>Rashōmon</i>

Le mot-clé **DESC** après la clause **ORDER BY** précise que nous voulons trier par ordre descendant. L'ordre ascendant, qui s'obtient via le mot-clé **ASC**, est le comportement par défaut.

LIMIT

Nous pouvons limiter le nombre de résultats. C'est par exemple utile pour une pagination, où pour chaque page nous pourrions limiter le nombre de résultat à quinze.

En l'occurrence, servons-nous ici du **LIMIT** pour ne récupérer que le film le plus récent de la table :

```
SELECT id, title FROM movies ORDER BY year DESC LIMIT 1
```

Résultat :

i d	title
6	<i>Rashōmon</i>

Récapitulons

Ce que nous venons de voir est la base d'une requête SQL « plate ». Récupérer les données d'une table, sous la forme d'un tableau qui sera ensuite exploitable par PHP sous la forme de tableaux associatifs ou d'objets. Nous reviendrons plus tard au pont avec PHP.

Pourquoi parlons-nous de requête plate ? Parce que nous n'extrayons ici les données que d'une seule table à la fois, sans avoir besoin des relations. Comment par exemple associer, dans un même résultat, les films et leurs réalisateurs ? Le chapitre suivant concernera les **jointures** entre les tables.

Voici un petit schéma résumant les différentes parties d'une requête SQL.

To choose columns	SELECT	name, MIN(year) AS oldest_work_from
To choose a table	FROM	artworks
To join a table	JOIN	artists
	ON	artworks.artist_id = artists.id
To filter records	WHERE	title != "The Mona Lisa"
To group records	GROUP BY	name
To filter groups	HAVING	MIN (year) < 1700
To sort output	ORDER BY	MIN(year);

Jointure

Reprenons notre base de données.

directors

i d	name	countr y
1	Michael Cimino	USA
2	Martin Scorsese	USA
3	Akira Kurosawa	Japon
4	Ingmar Bergman	Suède
5	Bruno Dumont	France

movies

i d	year	title	director_i d
1	1966	<i>Persona</i>	4
2	1997	<i>La Vie de Jésus</i>	5
3	2003	<i>Sarabande</i>	4
4	1978	<i>The Deer Hunter</i>	1
5	1976	<i>Taxi Driver</i>	2
6	1950	<i>Rashōmon</i>	3

Admettons que nous voulions la liste de tous les films par leurs titres, respectivement associés aux noms de leurs réalisateurs.

```
SELECT movies.title, directors.name
FROM movies
JOIN directors ON directors.id = movies.director_id
```


Notez que nous commençons à rédiger les requêtes sur plusieurs lignes, afin d'en améliorer la lisibilité.

Résultat :

title	name
<i>Persona</i>	Ingmar Bergman
<i>La Vie de Jésus</i>	Bruno Dumont
<i>Sarabande</i>	Ingmar Bergman
<i>The Deer Hunter</i>	Michael Cimino
<i>Taxi Driver</i>	Martin Scorsese
<i>Rashōmon</i>	Akira Kurosawa

Bon. Détaillons cette requête.

Premièrement, `movies.title`, `directors.name` au lieu de `title`, `name`. Nous n'avons pas eu besoin de cette syntaxe jusqu'ici parce qu'une seule table était impliquée dans la requête, mais nous pouvons – et devons parfois – préciser la table en plus de la colonne. Pourquoi ? Notre requête comporte plusieurs tables, qui peuvent souvent avoir des colonnes portant les mêmes noms. En l'occurrence la colonne `id`, mais ça peut tomber sur beaucoup de noms d'attributs très courants : `title`, `name`, `date`, etc. Dans un souci de levée d'ambiguïté, MySQL nous impose donc dans ces cas de préciser les tables.

Et enfin, la star : **JOIN**. Ce mot-clé nous permet de créer une **jointure** au sein d'une requête, et donc d'exploiter le potentiel des relations. Rappelons-le, la table `directors` et la table `movies` sont liées par une relation **One-to-Many**.

Traduisons notre requête SQL en français :

```
SELECT movies.title, directors.name
FROM movies
```

```
JOIN directors ON directors.id = movies.director_id
```

Sélectionne le titre de chaque film et le nom de chaque réalisateur ;

Ce depuis table des films jointe à la table des films, où le film sera associé

à son réalisateur ;

Comme la jointure est faite avec la condition selon laquelle l'`id` du réalisateur doit être égal pour chaque ligne au `director_id` du film, le résultat produit un tableau où chaque entrée est un film associé à son réalisateur respectif.

On pourrait préciser cette requête par l'ajout d'une clause **WHERE**.

```
SELECT movies.title, directors.name
FROM movies
JOIN directors ON directors.id = movies.director_id
WHERE directors.id = 4
```

Nous ne voulons ici que les films d'Ingmar Bergman. Résultat :

title	name
<i>Persona</i>	Ingmar Bergman
<i>Sarabande</i>	Ingmar Bergman

Un peu de tri ?

```
SELECT movies.title, directors.name
FROM movies
JOIN directors ON directors.id = movies.director_id
WHERE directors.id = 4
ORDER BY movies.year DESC
```

Les films d'Ingmar Bergman, du plus récent au plus ancien. Résultat :

title	name
<i>Sarabande</i>	Ingmar Bergman
<i>Persona</i>	Ingmar Bergman

Récapitulons

Notre dernière requête faisait cinq lignes. Ça monte vite. (:

Nous en resterons là pour ce cours théorique sur les requêtes de lecture et les jointures. Le SQL, c'est surtout une expérience qui s'acquiert avec le temps. Différents types de jointures, fonctions d'agrégation, clause **GROUP BY**, requêtes imbriquées, etc. Tout détailler ici ne ferait que dupliquer les nombreuses documentations qui existent sur le web. Tout un monde que vous allez découvrir par la pratique.

Le plus important est de bien en comprendre la logique. Ne laissez aucun angle mort à votre compréhension aux bases de la logique SQL, ne restez pas dans le

flou. Fondez votre pratique sur un beau « OK j'ai pigé ». Pour atteindre ce « OK j'ai pigé », posez des questions à vos formateurs, parcourez les **ressources** en lignes telle que **SQL.sh**. Puis pratiquez, pratiquez, pratiquez. À partir de maintenant, tous vos projets comporteront des bases de données, par la force des choses vous allez donc écrire beaucoup de requêtes SQL. La plupart seront très simples, mais certaines hanteront vos nuits !

Insert

Nous l'avons dit au début : les requêtes de lecture sont les plus velues.

Les requêtes d'écriture sont beaucoup plus simples et linéaires.

Considérons la table suivante :

directors

i d	name	countr y
1	Michael Cimino	USA
2	Martin Scorsese	USA
3	Akira Kurosawa	Japon
4	Ingmar Bergman	Suède
5	Bruno Dumont	France

Ajoutons une entrée à cette table.

```
INSERT INTO directors (name, country)
VALUES ("Julia Ducournau", "France")
```

Notre table se compose désormais d'une nouvelle entrée :

directors

i d	name	countr y
1	Michael Cimino	USA
2	Martin Scorsese	USA
3	Akira Kurosawa	Japon
4	Ingmar Bergman	Suède
5	Bruno Dumont	France
6	Julia Ducournau	France

Traduisons cette requête SQL en français :

Dans la table **directors**, ajoute une entrée dont les colonnes **name** et **country** prendront respectivement pour valeurs **Julia Ducournau** et **France** !

Notez au passage que nous n'avons pas besoin de préciser de valeur pour la colonne **id**. Cette colonne bénéficiera de ce que l'on appelle l'auto-incrément : MySQL calcule la valeur sans avoir besoin de notre intervention. Les clés primaires (ou IDs) sont souvent des clés numériques en auto-incrément, et bien que ce ne soit pas toujours le cas (UUIDv4 par exemple), c'est ce que vous verrez principalement au cours de cette formation. Vos formateurs vous montreront par la pratique comment ces réglages fonctionnent.

Il est également possible d'ajouter plusieurs entrées à la fois en une seule requête.

```
INSERT INTO directors (name, country)
VALUES
```

```
    ("Kenji Misumi", "Japon"),
    ("David Lynch", "USA"),
    ("Paul Verhoeven", "USA")
```

Nous avons ajouté trois entrées d'un coup.

directors

i d	name	countr y
1	Michael Cimino	USA
2	Martin Scorsese	USA
3	Akira Kurosawa	Japon
4	Ingmar Bergman	Suède
5	Bruno Dumont	France
6	Julia Ducournau	France
7	Kenji Misumi	Japon
8	David Lynch	USA
9	Paul Verhoeven	USA

Récapitulons

Voilà, l'insertion n'est pas beaucoup plus subtile que cela.

Non, vraiment, c'est tout !

Attendez... on me souffle que Paul Verhoeven n'est pas américain. Oups !
Rendez-vous au chapitre suivant pour corriger cette erreur.

Update

Une erreur s'est glissée dans notre table des réalisateurs au chapitre précédent :

directors

i d	name	countr y
1	Michael Cimino	USA
2	Martin Scorsese	USA
3	Akira Kurosawa	Japon
4	Ingmar Bergman	Suède
5	Bruno Dumont	France
6	Julia Ducournau	France
7	Kenji Misumi	Japon
8	David Lynch	USA
9	Paul Verhoeven	USA

Paul Verhoeven est un réalisateur néerlandais. Voyons ce qu'**UPDATE** peut faire pour nous.

```
UPDATE directors
SET country = "Pays-Bas"
WHERE directors.id = 9
```

Résultat :

directors

i d	name	country
1	Michael Cimino	USA
2	Martin Scorsese	USA
3	Akira Kurosawa	Japon
4	Ingmar Bergman	Suède
5	Bruno Dumont	France
6	Julia Ducournau	France
7	Kenji Misumi	Japon
8	David Lynch	USA
9	Paul Verhoeven	Pays-Bas

Traduisons la requête :

Dans la table **directors**, remplace la valeur de la colonne **country** par **Pays-Bas** pour la ou les entrées répondant à la condition d'avoir pour **id** la valeur **9** !

La clause **WHERE** permet ici de fixer une condition pour filtrer quelles entrées exactement on souhaite modifier. L'ID est bien sûr un attribut unique, et cette requête ne manipulera donc qu'une seule entrée ; mais l'**UPDATE** est une fonctionnalité puissante qui peut agir sur autant d'entrées qu'on le souhaite.

Considérons par exemple la requête suivante :

```
UPDATE directors
SET name = "Mickey Mouse", country = "Mars"
WHERE directors.country = "USA"
```

Si nous exécutons cette requête délirante, tous les réalisateurs des États-Unis seront renommés **Mickey Mouse** et vivront sur **Mars**. Pourquoi vous montrer un exemple idiot ? Pour vous faire comprendre que l'**UPDATE** est une requête puissante, irréversible, et donc potentiellement dangereuse si mal utilisée. Faites toujours très attention à l'exactitude chirurgicale de votre clause **WHERE**. De toute façon, en pratique dans le développement applicatif, les entrées à modifier sont la plupart du temps sélectionnées par l'ID. Il est assez rare de modifier plusieurs entrées à la fois. Mais ça peut arriver, donc méfiance, vous êtes prévenus, les données c'est la cour des grands, vous ne voulez pas faire une grosse bêtise dans la base de données d'un client. (:

Récapitulons

La requête **UPDATE** n'est pas beaucoup plus compliquée que la requête **INSERT**. On spécifie dans quelles colonnes on veut mettre quelles valeurs, en filtrant les entrées sur lesquelles agir via la clause **WHERE**. Simple !

Delete

La requête **DELETE** permet de supprimer une ou plusieurs entrées d'une table.

directors

i d	name	country
1	Michael Cimino	USA
2	Martin Scorsese	USA
3	Akira Kurosawa	Japon
4	Ingmar Bergman	Suède
5	Bruno Dumont	France
6	Julia Ducournau	France
7	Kenji Misumi	Japon
8	David Lynch	USA
9	Paul Verhoeven	Pays-Bas

Mettons que nous voulions supprimer David Lynch de notre table **directors** :

```
DELETE FROM directors  
WHERE directors.id = 8
```

Résultat :

directors

i d	name	country
1	Michael Cimino	USA
2	Martin Scorsese	USA
3	Akira Kurosawa	Japon
4	Ingmar Bergman	Suède
5	Bruno Dumont	France
6	Julia Ducournau	France
7	Kenji Misumi	Japon
9	Paul Verhoeven	Pays-Bas

David Lynch a disparu.

Et si nous souhaitons retirer tous les réalisateurs japonais ?

```
DELETE FROM directors  
WHERE country = "Japon"
```

directors

i d	name	country
1	Michael Cimino	USA
2	Martin Scorsese	USA
4	Ingmar Bergman	Suède
5	Bruno Dumont	France
6	Julia Ducournau	France
9	Paul Verhoeven	Pays-Bas

Akira Kurosawa et Kenji Misumi ont été supprimés.

Comme l'**UPDATE**, et à plus forte raison encore, la requête **DELETE** est très puissante et dangereuse. À moins de disposer de sauvegardes de la base de données, il n'existe pas de **Ctrl + Z**.

Chaque requête **DELETE** exécutée à la main sur une base de données en production doit vous faire transpirer par tous les pores de la peau. Ça rigole pas. Des erreurs dans un système peu solide, chez un client sans backups par exemple, peuvent avoir des conséquences graves.

Récapitulons

À ce stade, vous arrivez au bout des quatre grandes actions du CRUD :

- Create
- Read
- Update
- Delete

Nous avons vu que l'essentiel de la complexité logique se situait du côté des requêtes de lecture. Ces dernières peuvent parfois se révéler très alambiquées et difficiles à trouver en fonction des besoins, et nous n'avons même pas parlé d'optimisation. À côté, les requêtes de création, de modification et de suppression sont beaucoup plus simples. En sus, les requêtes de modification et surtout de suppression sont potentiellement très dangereuses : relisez bien ces requêtes avant de les exécuter, soyez sûrs de ce que vous faites.

Dans tous les cas, le SQL regorge de subtilités, d'approches différentes à un même problème, de syntaxes pouvant varier selon les préférences d'un développeur ou un autre. Ce support théorique n'a pas vocation à être absolument exhaustif, vous allez devoir faire vos recherches, poser vos questions, étudier et pratiquer.

Exercice - 3WA Trade

Votre client, l'entreprise de commerce 3WA Trade, vous appelle dans la panique.

À cause d'une injection SQL oubliée par l'un de vos confrères aujourd'hui devenu pâtissier et ayant changé de numéro de téléphone, leur base de données a été totalement effacée. Le site fonctionne toujours, mais les pages sont vides.

Avant de corriger la faille, vous essayez de vous connecter en urgence à leur serveur et vous apercevez avec effroi que vous n'y avez plus accès. Un upload de fichier malveillant dans un formulaire mal protégé aura permis au même pirate de prendre le contrôle du serveur.

Ah ! Ça y est, le client vous informe qu'il vient de recevoir un e-mail d'un certain Dimitri. Celui-ci, honnête et courtois pirate biélorusse faisant son petit bonhomme de chemin dans une vie froide et cruelle, leur demande un versement de 2 bitcoins s'ils veulent revoir un jour leur serveur. Pas tant que ça, hein ? Ça équivaut à près de 100 000 € au moment du hack. (:

Tandis que l'équipe technique tente désespérément de reprendre le contrôle du serveur et que le PDG de la 3WA Trade caresse tristement, du bout des doigts, le cuir de son porte-feuilles, une mission importante vous est assignée.

En attendant que la situation s'améliore, votre client ne peut pas se permettre de stopper son activité, aussi vous demande-t-il d'extraire rapidement des données d'un ancien dump SQL.

Importez le fichier `3watrade.sql` dans votre base de données et attellez-vous à traduire ce mail de votre client en requêtes SQL :

Salut,

C'est un peu le feu ici au bureau ! Merci pour ta réactivité !

Alors... il nous faut d'urgence :

1. La liste de tous les utilisateurs ;
2. La liste de tous les utilisateurs rangée par noms de famille ;
3. Le dernier utilisateur inscrit ;
4. La liste de tous les utilisateurs fêtant leur anniversaire ce mois-ci ;
5. Le nombre total d'utilisateurs ;

6. La liste de tous les utilisateurs associés à leurs villes respectives ;
7. La liste de tous les utilisateurs vivant à une adresse sans numéro ;
8. La liste de tous les produits dont le prix est supérieur à 1 000 € ;
9. La liste des photos du produit le moins cher ;
10. La liste de tous les produits appartenant à la catégorie « Voyage » ;
11. La liste de tous les utilisateurs ayant effectué plus de dix commandes ;
12. La liste de tous les produits achetés par le premier utilisateur inscrit.

Cordialement,

Magali

Conseils d'installation

- Ouvrez l'export SQL sur Moodle ;
- Copiez la totalité du contenu de l'export (Ctrl+A, Ctrl+C) ;
- Sur **PHPMyAdmin**, aller dans l'onglet SQL, coller cette immense requête dans le champ de texte ;
- Modifier les deux occurrences de **yourlogin** par votre propre identifiant 3WA, lignes 22 et 24 ;
- Exécuter la requête et patienter.

L'import peut mouliner pendant plusieurs longues minutes, six ou sept parfois. Pas d'inquiétude, faites-vous un café. À l'issue, l'essentiel est que votre base soit bien créée avec ses huit tables. Une erreur 504 peut même apparaître à la fin, souvent elle est inoffensive et les tables ont été créées malgré tout. Dans tous les cas, les formateurs sont là pour vous accompagner dans vos premiers pas avec MySQL et **PHPMyAdmin**.

